Hochat Online

Where Each WMA4 Lecture Topic Is Used in Our System · Group 2

WMA4 – Advanced Web Development · Tarlac State University, College of Computer Studies
Lectures: Week 2–3 PHP & SQL · Week 4–5 Object-Oriented PHP · Week 6–7 MySQL · Topic 4 Auth & Security · Topic 5 REST API

The lectures teach these topics in **plain PHP** (PDO, `$_POST`, `password_hash()`). Hochat Online does the same things with **Laravel**, which is the "Framework migration" promised in our paper. In the defense: "*We learned it in plain PHP, and here is the Laravel version in our system.*"

Week 2–3: Intro to PHP and SQL

Lecture topic	Where it is used in our system
Variables, arrays, `foreach`, conditions	Everywhere, e.g. the cart array `[menu_item_id => quantity]` in `CartController.php`
Functions, return types	Every controller method, e.g. `public function store(...): RedirectResponse`
Anonymous / arrow functions, callbacks	`fn (MenuItem $item) => ...` in `Order::placeFor()` and `CartController::index`
Constants	`Order::DELIVERY_FEE = 50`, `Order::DELIVERY_AREA = 'Gerona'`
Forms, `$_GET` / `$_POST`	`$request->validated(...)` (Laravel's safe version of `$_POST`); menu search and report dates use GET
PHP–MySQL connection	`.env` (`DB_DATABASE=hochat_online`); Laravel connects for us
SQL CRUD	Menu items: add, edit, sold-out, delete (`Admin/MenuItemController.php`)
Prepared statements	**Eloquent** uses prepared statements automatically; no raw SQL strings with user input

Week 4–5: Modern Object-Oriented PHP

Lecture topic	Where it is used in our system
Classes and objects	`app/Models/User.php`, `Order.php`, `MenuItem.php`, every controller
Constructors, encapsulation	`$fillable` controls which fields may be set; `protected function casts()`
Inheritance	`class Order extends Model`, `class CartController extends Controller` (abstract class)
Polymorphism	`Order`, `MenuItem` and `User` all inherit `create()`, `update()` and `delete()` from `Model`
Traits	`use HasApiTokens, HasFactory, Notifiable;` in `User.php`

Type declarations	All methods have parameter and return types; `enum Role: string` in `app/Enums/Role.php`
Exception handling	`ValidationException` thrown in `Order::placeFor()`; `abort(403)` in `EnsureUserHasRole`
Namespaces	`namespace App\Http\Controllers\Admin;` etc.
Composer and autoloading	`composer.json` → `"App\\": "app/"` (PSR-4)
PSR-12 coding standard	**Laravel Pint** (`pint.json`) formats the code automatically
Interfaces	⚠ **None written by the group** (see Gaps)

Week 6–7: MySQL Database

Lecture topic	Where it is used in our system
Relational concepts, tables	5 tables: users, categories, menu_items, orders, order_items
Data types	Migrations: `string(100)`, `decimal(8,2)`, `enum`, `boolean`, `dateTime`
Keys and constraints	`foreignId(...)->constrained()`, `email->unique()`, `cascadeOnDelete()` in `database/migrations/`
CREATE DB / CREATE TABLE (DDL)	`php artisan migrate` runs the migrations
INSERT, multiple records	`MenuSeeder.php` inserts 85 items; `Order::placeFor()` inserts the order and its items
Last inserted ID	`$order->id` right after `create()`, used to link the `order_items`
SELECT, WHERE, ORDER BY, LIMIT	`->where('status', ...)`, `->latest()`, `->limit(50)` in `Admin/DashboardController.php`
UPDATE / DELETE	Order status update; cancel order; delete menu item
TCL (transactions)	`DB::transaction(...)` in `Order::placeFor()`: the order and its items save together or not at all
SQL injection	Prevented by Eloquent's prepared statements

Topic 4: Authentication, Authorization and Backend Security

Lecture topic	Where it is used in our system
Users table design	`users` migration with `email` unique, `password`, `role`
Registration flow	`Auth/RegisterController.php` + `RegisterRequest.php`
Login flow	`Auth/LoginController.php`: `Auth::attempt(...)`
`password_hash()` / `password_verify()`	`'password' => 'hashed'` in `User.php` (bcrypt, same as `password_hash`); `Hash::check()` in the API login
Session authentication	Laravel sessions; `session()->regenerate()` after login (stops session fixation)
Protecting a page	`auth` middleware in `routes/web.php`
Authorization (roles)	`role:` middleware → `EnsureUserHasRole.php`; `OrderPolicy.php` (own orders only)
CSRF tokens	Automatic on every form (Laravel + Inertia)
Secure logout	`session()->invalidate()` + `regenerateToken()` in `LoginController::destroy`
Brute-force protection	`throttle:5,1` on API login and forgot-password
Token authentication	Sanctum: `createToken('api')` in `Api/V1/AuthController.php`
Session vs token	The website uses **sessions** and the API uses **tokens**, so our project shows both

Topic 5: PHP REST API

Lecture topic	Where it is used in our system
REST principles, resource URLs	`routes/api.php`: `/api/v1/menu`, `/cart`, `/orders/{order}`
HTTP methods	GET / POST / PATCH / PUT / DELETE routes in `api.php`
Status codes	`201` on create and `204` on delete (`Api/V1/Admin/MenuItemController.php`); `401`, `403`, `404`, `422` handled by Laravel
`json_encode()` responses	**API Resources**: `MenuItemResource`, `OrderResource` (`app/Http/Resources/`)
Reading JSON (`php://input`)	`$request->validated()` reads JSON bodies automatically
Routing to one entry point	`public/index.php` → `routes/api.php`
Validation before writing	`app/Http/Requests/Api/*Request.php`
Standardized error responses	`bootstrap/app.php` makes all `/api` errors JSON
Bearer token check and roles	`auth:sanctum` + `role:Staff,Administrator` middleware
Secure file upload	Menu photo: `'photo' => ['nullable', 'image', 'max:2048']` in `MenuItemRequest.php`, saved with `->store('menu', 'public')`
Postman testing	`Hochat-Online.postman_collection.json` (32 requests) + Pest API tests in `tests/Feature/ApiTest.php`
PDO repository, `try-catch` / `PDOException`	⚠ Not written by hand: Eloquent replaces PDO, and Laravel handles database errors

Gaps to know about

The paper's "Expected Final-Term Technical Implementation" section promises both of these:

1. **Interfaces:** the paper says *"a shared Role interface or inheritance."* We have inheritance and a `Role` enum, but **no interface written by the group**.
2. **try-catch:** the paper says *"try-catch exception handling for database and validation errors."* There is **no** `try { } catch` in our code; Laravel catches errors centrally.

> **Defense answer if asked:** "Laravel handles database exceptions centrally in `bootstrap/app.php`, and we throw `ValidationException` for business rule errors."